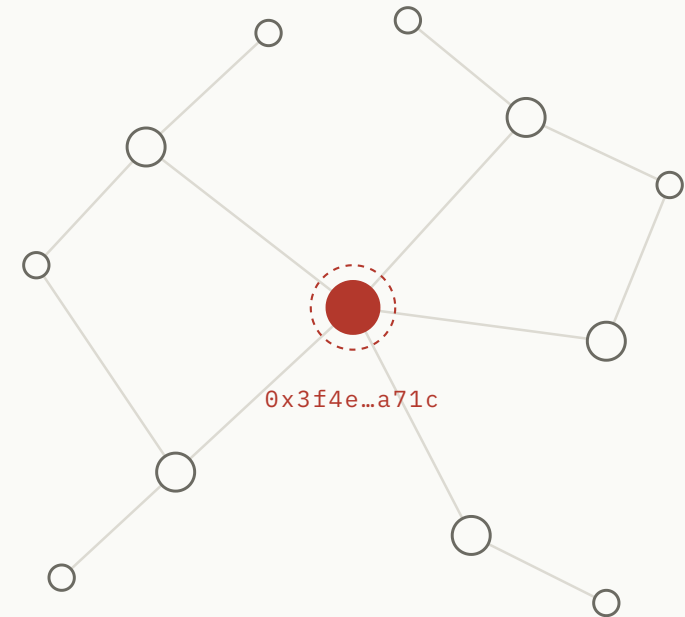


Detecting Illicit Ethereum Transactions with Graph-Derived Features

*Can an address's network of connections reveal
fraud that its own features cannot?*

Sanskriti Acharya & Sweta Sharma

`eth-illicit-detection.streamlit.app`



Is this address a scam?

Every Ethereum transfer is public and permanent — and there are over a million a day. No investigator can read them all.

Track A SUPERVISED · BINARY	1 address IN · PHISHING OR SCAM?	a probability OUT · NEVER A VERDICT
---------------------------------------	--	---

Built for an analyst triaging addresses, so the output is a review priority. The deliverable is a short, ranked list.

Accuracy is unusable here

0.845

ACCURACY OF A MODEL THAT ALWAYS ANSWERS “LICIT”

$F_1 = 0.000$

THE SAME MODEL, ON THE METRIC THAT MATTERS

$F_1 = \frac{2 P R}{P + R}$ on the illicit class, reported with PR-AUC

A missed scam lets fraud continue; a false alarm costs an analyst minutes. Recall matters more — but F_1 only rises when both are respectable.

Only the labels are borrowed

2,852

LABELLED ADDRESSES

652

ILLICIT · 22.9%

2017–18

BOTH CLASSES, SAME ERA

100%

FEATURES COMPUTED BY US

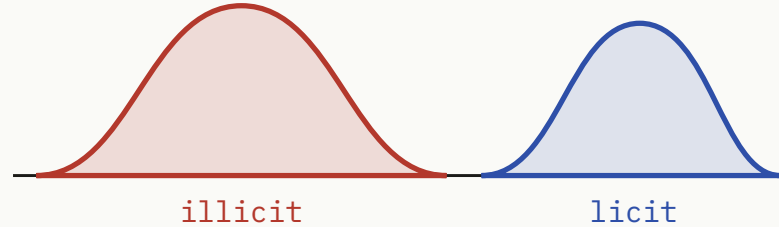
Labels — *the MyEtherWallet darklist of reported scam addresses, deduplicated 715 → 652 wallets.*

Everything else — *raw transactions fetched address-by-address from Blockscout. No pre-cleaned download exists.*

The shortcut we had to close

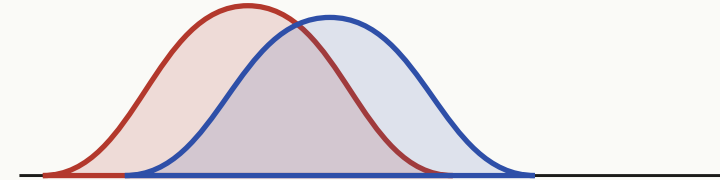
97% of the darklist dates to 2017–18. Sample licit addresses from recent blocks and `block_number` alone separates the classes perfectly.

NAIVE: LICIT FROM RECENT BLOCKS



separable by block number alone ✗

ERA-MATCHED (OURS)



distributions overlap — shortcut closed ✓

block number →

Fig. 1 — same logic twice more: contracts excluded from both classes, absolute block numbers excluded from features.

29 behavioural features, plus 7 from the graph

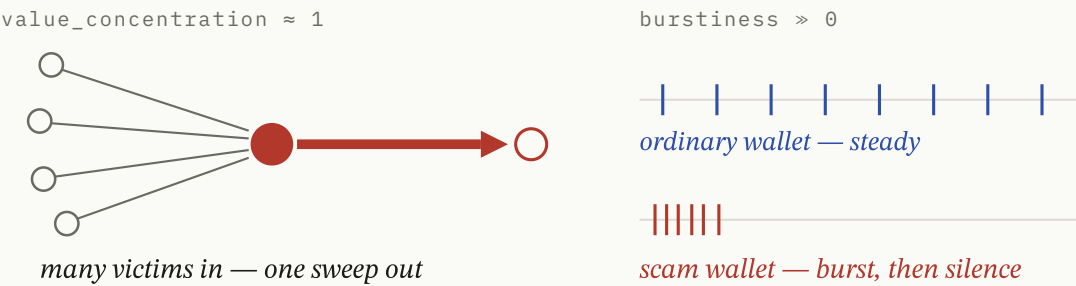


Fig. 2 — two features that encode a fraud pattern. Graph features: degree, in/out degree, PageRank, clustering, community size, neighbour risk.

715 → 652 labels	<i>one wallet, several campaigns — duplicates would span the split</i>
empty addresses kept	<i>deleting them shifts class balance and hides a real case</i>
gasUsed dropped	<i>the API returns 0 on many real transfers</i>
scaler in pipeline	<i>the app cannot preprocess differently than training did</i>

The split is temporal, not random

Scam operations run clusters of sibling wallets. A random split puts siblings on both sides, and the model recognises the cluster instead of generalising.

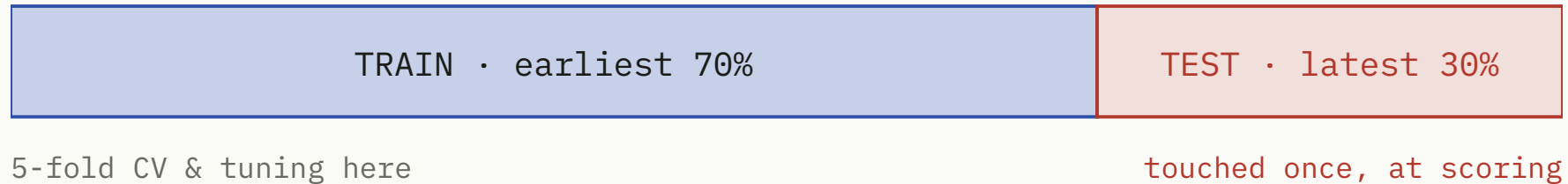


Fig. 3 – addresses ordered by first on-chain appearance. Learn from what is known; score what came later.

The feature that could have leaked

`neighbour_risk_ratio` asks “what fraction of my labelled neighbours are illicit?” Computed over *all* labels, it leaks the answer outright.

So `graph.py` takes visible labels as an argument, and training passes **the training split’s labels only**.

PINNED BY UNIT TESTS – including that an address can never read its own label back through a self-loop.

Four tiers, one experiment

TIER 1 **Majority class.** *always “licit” — the baseline to beat and report*

TIER 2 **Behaviour only.** *logistic regression & random forest, 29 features*

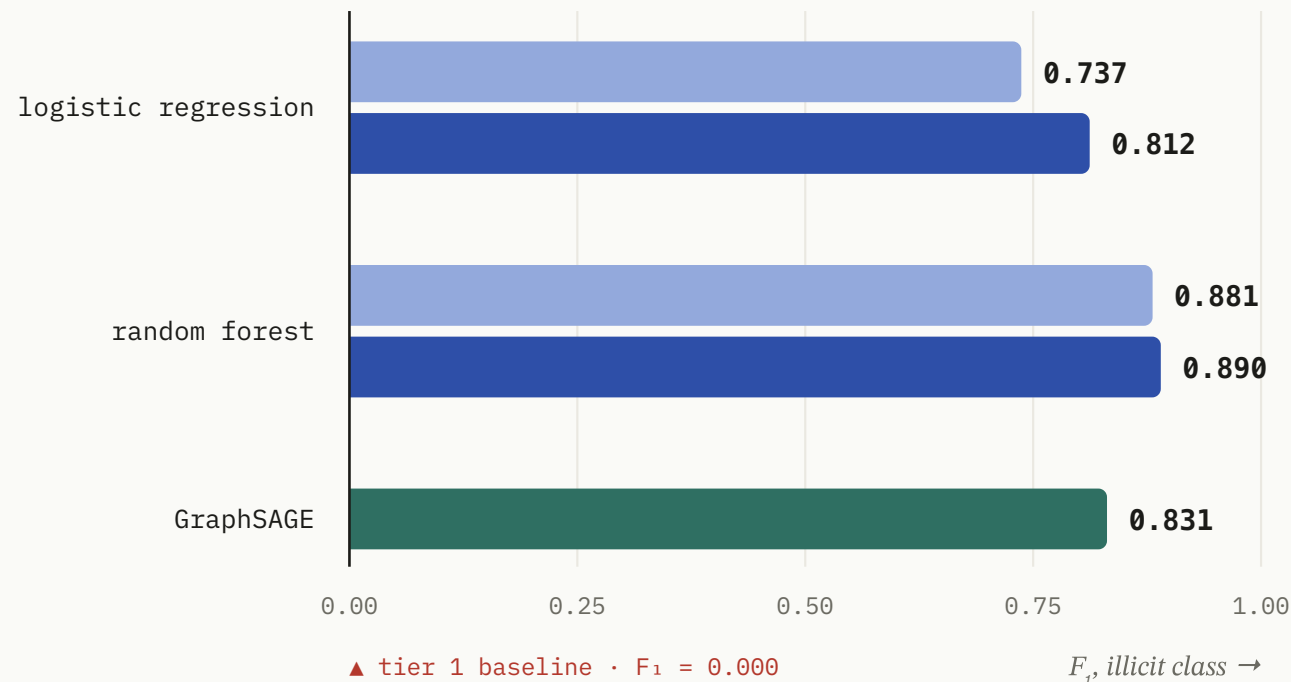
TIER 3 **Behaviour + graph.** *the same two models; only the feature set changes*

TIER 4 **Graph neural network.** *GraphSAGE on the raw address graph — no hand-made features*

CLASS-WEIGHTED · 5-FOLD CV ON THE TRAINING SPLIT ONLY · ONE TEMPORAL SPLIT THROUGHOUT

Nothing changes but the feature set

behaviour + graph features raw graph



$F_1 = 0.890$

SHIPPED · RF + GRAPH

129 of 133

SCAMS CAUGHT ON TEST

28

FALSE ALARMS · THE REVIEW QUEUE

Fig. 4 – F_1 on the illicit class, held-out temporal test split (856 addresses, 133 illicit).

Does the graph help?

Yes — but modestly, and it depends on the model.

+0.075

F_1 · LOGISTIC REGRESSION

A LINEAR MODEL CANNOT BUILD THIS ITSELF

+0.009

F_1 · RANDOM FOREST

+0.017 PR-AUC

≈9%

OF FOREST IMPORTANCE

OUT_DEGREE, COMMUNITY_SIZE, PAGERANK

We would rather report +0.009 accurately than round it into a story. Our graph is one hop deep — this is a floor, not a ceiling.

What if the model reads the graph itself?

*A two-layer **GraphSAGE** over all 44,432 nodes — including unlabelled neighbours — given no graph features at all. It has to rediscover what network context means by passing messages along the edges.*

SAME TEMPORAL SPLIT · SCALER ON TRAINING ROWS ONLY
neighbour_risk_ratio withheld — feeding it our answer would make the comparison circular.

*It loses to the forest on precision — 1,996 labelled training nodes is a small corpus for learned features. **That the forest still wins is evidence our feature engineering wasn't leaving much behind.***

$F_1 = 0.831$

TIER 4 · GRAPHSAGE

0.884

PR-AUC

128 of 133

SCAMS CAUGHT · 96% RECALL

Is the signal real?

*Licit addresses were sampled from random blocks, so busy addresses are over-picked: **61%** hit the 100-transaction cap, against **8%** of illicit ones.*

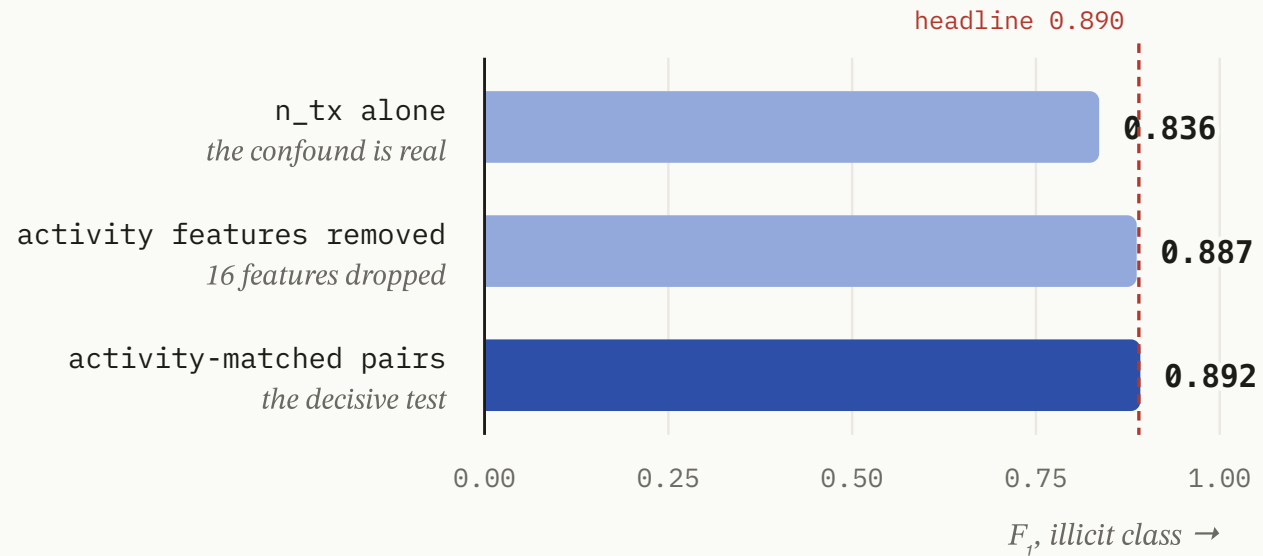
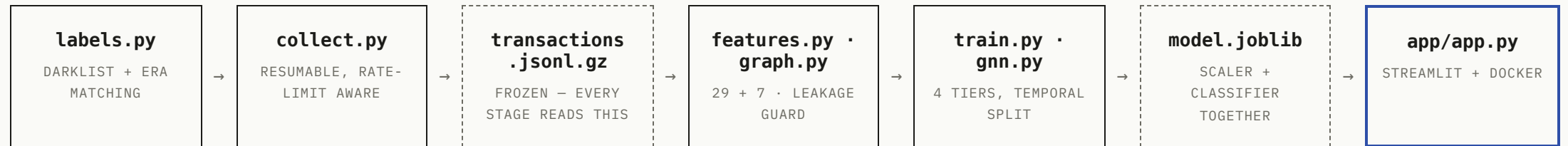


Fig. 5 – within activity-matched pairs, activity carries no information by construction – and F_1 does not drop. **The confound is genuine but not load-bearing.**

One path from raw chain to application



NO DUPLICATED PREPROCESSING

The app imports `src/features.py` rather than reimplementing it — the classic way a deployed model silently diverges from the one that was evaluated.

TESTS WHERE BUGS HIDE

Cursor construction, feature arithmetic on hand-built transactions, the leakage guard, the saved scaler.

A probability and a priority — never a verdict

-
- TAB 1 **Look up an address.** *fetches live, scored by the saved pipeline*
-
- TAB 2 **Explore features by hand.** *presets like “collect-and-sweep”, no network needed*
-
- TAB 3 **Model performance.** *all four tiers, importances, graph statistics*
-

eth-illicit-detection.streamlit.app

LIVE NOW — SWITCHING TO THE APP

The footer states it plainly: a LICIT score means “not reported”, not “cleared”. A flag is a candidate for review, not a suspect.

Task division & further work

Sanskriti

- Blockscout client & collector
- labels & era-matched sampling
- EDA notebook
- Streamlit app & deployment

Sweta

- behavioural & graph features
- leakage guard, unit tests
- four-tier training & the GNN
- sensitivity analysis

Further work

- crawl one hop deeper
- more positives — add OFAC
- recalibrate to the true base rate
- measure rate limits before designing around them

CAN AN ADDRESS'S NETWORK REVEAL FRAUD ITS OWN FEATURES CANNOT?

Yes — decisive for a linear model,
marginal for a forest, and confirmed by a
network that reads the raw graph without
beating the features we built by hand.

SANSKRITI ACHARYA & SWETA SHARMA · ETH-ILLCIT-DETECTION.STREAMLIT.APP